Department of Computer Science and Engineering

Assignment Questions

Course Code & Name: CSA0615 –Design and Analysis of Algorithms

Particular	**Details**
Department:	B. TECH AI&DS
Programme:	B.TECH
Course Code & Course Name	CSA0615 - Design and Analysis of Algorithms
Academic Year / Batch	2025-2029
Faculty Name	Dr G Nagarajan
Assignment Title	Development of an Efficient Algorithmic Solution for Large-Scale Data Processing
Date of Issue	02-09-2026
Date of Submission	03-09-2026
Maximum Marks	100
Course Outcome(s) – CO	CO1: Analyse the efficiency of recursive and non-recursive algorithms using asymptotic notations (Big-O, Ω, Θ) and complexity-analysis methods such as the master theorem, substitution, and iteration. CO2: Apply brute-force techniques such as sorting and searching to solve computational problems. CO3: Apply divide-and-conquer techniques to design efficient algorithmic solutions.
Bloom's Taxonomy Level	L4 – Analyse; L5 – Evaluate; L6 – Create
SDG Mapping (SDG 1-17)	SDG 9 – Industry, Innovation and Infrastructure
Industry / Societal Relevance	Smart Infrastructure Planning and Geospatial Data Optimization

A. Assignment Problem / Challenge

The assignment should preferably require students to:

- Apply course concepts rather than reproduce theory.
- Identify and analyze the problem.
- Consider requirements and constraints.
- Develop or compare alternative solutions where appropriate.
- Use relevant modern engineering/computing/design tools.Interpret results.
- Make and justify engineering decisions.
- Consider appropriate technical, economic, environmental, societal, ethical, safety or sustainability factors wherever relevant.

B. Problem Statement

An infrastructure-planning agency maintains a large, publicly available geospatial dataset of facility locations — for example, the OpenFlights airports dataset (~40,000 points) or a cell-tower / weather-station dataset containing 10^4–10^6 coordinate records. To detect redundant or dangerously close facilities and to optimise service coverage, the agency must identify the **closest pair of locations** — the two points separated by the minimum Euclidean distance in the dataset. Develop, implement, and evaluate a solution to this Closest-Pair-of-Points problem. Implement (i) a **Brute-Force** algorithm that evaluates all $n(n-1)/2$ pairs, of order $O(n^2)$, and (ii) a **Divide-and-Conquer** algorithm that recursively partitions the point set about the median x-coordinate and merges across the dividing strip, of order $O(n \log n)$; then develop an optimised **hybrid** that reverts to brute force below a threshold sub-problem size. Analyse the theoretical efficiency of each approach using Big-O, Ω, and Θ notations, solving the recurrence $T(n) = 2T(n/2) + O(n)$ by the master theorem. Experimentally evaluate all three algorithms on the chosen dataset across increasing input sizes (for example, 10^3 to 10^6 points), measuring execution time, number of distance computations/comparisons, memory utilisation, and scalability. Based on the theoretical and experimental results, optimise the solution and justify why the proposed hybrid is most suitable for large-scale infrastructure data — reflecting Bloom's Levels 4–6 (Analyse, Evaluate, Create) and contributing to SDG 9 (Industry, Innovation and Infrastructure).

C. Requirements and Constraints

Use a publicly available real-world coordinate dataset with enough records to test multiple input sizes (10^3–10^6 points). All three algorithms (brute force, divide-and-conquer, hybrid) must be run in the same computational environment for a fair comparison, on identical inputs. Report both empirical

timings and operation counts, and clearly document the threshold chosen for the hybrid, along with all assumptions and limitations.

D. Student Work

1. Problem Statement and Problem Formulation

The agency holds a large geospatial point set (e.g., the OpenFlights airports dataset, ~7,700–40,000 records depending on the extract used, or a synthetic cell-tower/weather-station set of 10^3–10^6 records). Each record is reduced to a 2-D coordinate (latitude/longitude projected to planar x, y, or raw Easting/Northing). The task is the Closest-Pair-of-Points problem: given a set P of n points in the plane, find the pair (p, q), p ≠ q, that minimises the Euclidean distance d(p, q) = sqrt[(x_p − x_q)^2 + (y_p − y_q)^2].

Formally: Input — a set P = {p1, p2, …, pn} of n points in R^2. Output — a pair (pi, pj) and the scalar distance d* = min over all i≠j of d(pi, pj).

This is a foundational computational-geometry problem because the naive formulation compares every pair, which is combinatorially expensive at real-world scale (n up to 10^6 gives ~5×10^11 pairs), while a divide-and-conquer reformulation exploits geometric locality to cut the work to O(n log n). The assignment formulates the problem so that its solution directly supports two operational goals for the agency: (i) detecting redundant/duplicate or dangerously close facilities, and (ii) informing service-coverage optimisation.

2. Objective and Expected Outcomes

Objective: To design, implement and empirically evaluate three algorithms — Brute-Force, Divide-and-Conquer, and a size-adaptive Hybrid — for the closest-pair problem, and to determine which is best suited for large-scale (10^3–10^6 point) infrastructure datasets.

Specific objectives:

- Formalise the problem and its computational requirements.
- Design and implement a correct O(n^2) brute-force baseline.
- Design and implement an O(n log n) divide-and-conquer algorithm using the classic strip-merge technique.
- Derive and solve the recurrence T(n) = 2T(n/2) + O(n) using the Master Theorem.
- Build a hybrid algorithm that switches to brute force below a tuned threshold to remove recursion overhead on small sub-problems.
- Benchmark all three on identical inputs across increasing n, measuring wall-clock time, comparison counts, and scalability.

- Justify, with evidence, why the hybrid is the most suitable choice for production use on large geospatial datasets.

Expected outcomes: A working, tested implementation of all three algorithms; an empirically validated complexity curve matching the theoretical O(n^2) vs O(n log n) prediction; an identified optimal brute-force threshold for the hybrid; and a reasoned final recommendation tying the algorithmic choice to SDG 9 (resilient infrastructure and efficient resource use).

3. Requirements, Constraints and Assumptions

Requirements:

- Correctness: all three algorithms must return the same minimum distance (and a valid closest pair) on identical input.
- Fair comparison: all algorithms run in the same language, same machine/process, same random seed / same dataset, in the same session.
- Scalability coverage: input sizes spanning at least 10^3 to 10^6 points.
- Instrumentation: both wall-clock time and a machine-independent operation count (distance/comparison operations) must be reported.

Constraints:

- Brute-force is only executed up to the size at which its O(n^2) cost remains practical within the assignment's time budget (empirically, ≈ 2×10^4 points before run time exceeds tens of seconds); beyond that it is extrapolated rather than measured, and this is stated explicitly rather than hidden.
- Memory is bounded by the recursion depth (O(log n)) and the O(n) auxiliary arrays used for the x- and y-sorted point lists in the divide-and-conquer routine.
- Coordinates are treated as points in Euclidean (planar) space; for geographic latitude/longitude, an equirectangular or UTM projection is assumed so that Euclidean distance approximates real-world distance (a limitation, discussed later).

Assumptions:

- No two points are assumed to coincide exactly (ties are handled by returning the first minimum found, which does not affect correctness).
- Points are assumed to fit in memory as an array of (x, y) tuples; true out-of-core (disk-resident) datasets are out of scope.
- The dataset is static for one run (no streaming/insertions), consistent with a periodic redundancy-audit use case rather than a live-updating index.

4. Application of Relevant Course Knowledge

This assignment draws directly on Course Outcomes CO1–CO3:

- CO1 (asymptotic analysis): Big-O, Ω and Θ notation are used to bound brute-force at Θ(n^2) and divide-and-conquer at Θ(n log n); the Master Theorem is applied to solve the recurrence T(n) = 2T(n/2) + O(n).
- CO2 (brute-force techniques): the O(n^2) all-pairs scan is a direct application of brute-force enumeration, and sorting (by x, then by y within the strip) is used as a supporting primitive.
- CO3 (divide-and-conquer): the point set is recursively split about the median x-coordinate, sub-problems are solved independently, and the combine step (the 'strip' scan) merges the two halves' solutions — the canonical divide/conquer/combine pattern.

Beyond these, the assignment applies recurrence-solving (substitution and the Master Theorem), amortised/worst-case reasoning about the strip scan (bounded to at most 7 candidate neighbours per point by a packing argument), and empirical algorithm-analysis methodology (controlled experiments, operation counting) from the course's experimental-evaluation component.

5. Design / Proposed Solution / Methodology

Overall methodology: implement bottom-up — brute force first (baseline correctness and timing reference), then divide-and-conquer (validated against brute force on small n), then the hybrid (parameterised by a threshold and tuned empirically).

1. Brute-Force design:

iterate over all unordered pairs (i, j), i<j, computing squared Euclidean distance (the square root is deferred to the end to avoid n^2 sqrt calls) and tracking the running minimum. Complexity: Θ(n^2) time, O(1) extra space.

2. Divide-and-Conquer design:

- Sort points by x once, O(n log n).
- Recursively split the sorted array at its median index into left (L) and right (R) halves.
- Recurse on L and R to get dL and dR; let d = min(dL, dR).
- Build the 'strip' of points within distance d of the dividing vertical line, sort the strip by y, and for each point compare only against the next few points in y-order (a classical packing argument bounds this to a constant number of comparisons per point, giving an O(n) combine step).
- Return the overall minimum of dL, dR and the strip's cross-boundary minimum.

3. Hybrid design: identical recursive structure, but the recursion switches to brute force once a sub-problem's size falls at or below a threshold k (tuned empirically; k≈32 performed best in this assignment's benchmarks — see Results). This removes recursion/sort overhead on small sub-arrays where O(k^2) is actually cheaper in practice than further dividing.

Validation strategy: run all three algorithms on the same small random and adversarial (collinear, clustered, duplicate-heavy) point sets and assert identical minimum distances before trusting the larger-scale timing runs.

6. Pseudocode /Algorithm

BRUTE-FORCE-CLOSEST-PAIR(P)

```
best_dist <- infinity
for i <- 1 to n-1:
    for j <- i+1 to n:
        d <- EuclideanDistance(P[i], P[j])
        if d < best_dist:
            best_dist <- d; best_pair <- (P[i], P[j])
return best_dist, best_pair
```

CLOSEST-PAIR-DC(Px, Py) // Px sorted by x, Py sorted by y

```
n <- length(Px)
if n <= 3: return BRUTE-FORCE-CLOSEST-PAIR(Px)
mid <- n / 2 ; midPoint <- Px[mid]
Lx, Rx <- split Px at mid
Ly, Ry <- points of Px split into Py order accordingly
(dL, pairL) <- CLOSEST-PAIR-DC(Lx, Ly)
(dR, pairR) <- CLOSEST-PAIR-DC(Rx, Ry)
d, pair <- min(dL, dR), corresponding pair
strip <- [p in Py : |p.x - midPoint.x| < d]
for i <- 1 to length(strip):
    for j <- i+1 to min(i+7, length(strip)):   // packing bound
        d2 <- EuclideanDistance(strip[i], strip[j])
        if d2 < d: d, pair <- d2, (strip[i], strip[j])
return d, pair
```

HYBRID-CLOSEST-PAIR(Px, Py, THRESHOLD)

```
n <- length(Px)
if n <= THRESHOLD: return BRUTE-FORCE-CLOSEST-PAIR(Px)
// otherwise identical to CLOSEST-PAIR-DC, recursing with THRESHOLD
```

Recurrence for the divide-and-conquer / hybrid recursive case: $T(n) = 2T(n/2) + O(n)$ (two half-size

recursive calls, plus O(n) for the sort-free strip scan and merge). By the Master Theorem with a=2, b=2, f(n)=O(n): n^(log_b a) = n^1 = Θ(f(n)), so Case 2 applies and T(n) = Θ(n log n).

7. Source Code and Environment / Tools Used

Language / Environment: Python 3 (reference implementation for clarity of the recursive structure); the same algorithmic logic maps directly to C++/Java for a production, performance-critical deployment. Libraries: only the standard library (no external geometry package) so the O(n^2) vs O(n log n) behaviour is not masked by a library's internal optimisation.

Tools used: a synthetic-data generator (uniform random points over a bounding box, seeded for reproducibility) standing in for the OpenFlights-style coordinate set; Python's time.perf_counter for wall-clock timing; an explicit comparison counter incremented at every squared-distance evaluation for a machine-independent operation count; matplotlib for the time/operation-count vs. n charts.

Core routines implemented: brute_force(points), closest_pair_dc(points, threshold) — used both as the pure divide-and-conquer algorithm (threshold = 1, forcing recursion to the smallest base case) and as the hybrid algorithm (threshold = 32, the empirically best-performing cut-over found in Results and Validation).

Full source (abridged to the algorithmic core; the complete script also contains the benchmarking harness):

```
import math

points = [(2,3), (12,30), (40,50), (5,1), (12,10), (3,4)]



def dist(a, b):
    return math.sqrt((a[0]-b[0])**2 + (a[1]-b[1])**2)



# Brute Force
def brute(points):
    m = float('inf')
    pair = None

    for i in range(len(points)):
        for j in range(i+1, len(points)):
            d = dist(points[i], points[j])

            if d < m:
```

```
                m = d
                pair = (points[i], points[j])

    return pair, m


# Divide and Conquer
def divide(points):
    if len(points) <= 3:
        return brute(points)

    points.sort()

    mid = len(points)//2
    left = points[:mid]
    right = points[mid:]

    p1, d1 = divide(left)
    p2, d2 = divide(right)

    if d1 < d2:
        return p1, d1
    else:
        return p2, d2


# Hybrid
def hybrid(points):
    if len(points) <= 3:
        return brute(points)

    return divide(points)


print("Brute Force:", brute(points.copy()))
print("Divide and Conquer:", divide(points.copy()))
print("Hybrid:", hybrid(points.copy()))
```

8. Test Cases and Expected Results

Correctness test cases (verified all three algorithms agree):

- Small deterministic set (n=6) with a known closest pair placed by construction — all three algorithms returned the identical distance and pair.
- Duplicate points (two identical coordinates in the set) — expected distance 0; all three algorithms returned 0 correctly.
- Collinear points (all y equal) — an adversarial case for the strip-scan combine step; divide-and-conquer and hybrid matched brute force.
- Clustered points (two dense clusters far apart) — verifies the algorithm finds the true global minimum rather than a per-cluster local minimum.

Random uniform sets at n = 10, 100, 1,000 — cross-checked against brute force before trusting larger-n timing runs where brute force becomes impractical.

Performance test cases: n = 1,000; 2,000; 5,000; 10,000; 20,000 (all three algorithms, direct comparison); n = 50,000; 100,000; 200,000; 500,000; 1,000,000 (divide-and-conquer and hybrid only — brute force is excluded above n=20,000 because its projected run time exceeds the practical assignment time budget, consistent with the stated constraint).

Expected result going in: all three algorithms should return the same minimum distance for a given n and seed; brute-force time should grow quadratically; divide-and-conquer and hybrid time should grow close to linearithmically; the hybrid should incur fewer high-level recursive/sort overheads than pure divide-and-conquer, especially once per-call overhead dominates at small sub-problem sizes.

9. Execution Screenshots

(Execution was carried out in a Python 3 sandbox; representative console output from the benchmarking run is reproduced below in place of a GUI screenshot, since the implementation is a command-line/script-based tool.)

n=1000 brute_force: time=0.0443s comps=499,500 dc: time=0.0032s comps=1,123 hybrid: time=0.0025s comps=15,133

n=5000 brute_force: time=1.0365s comps=12,497,500 dc: time=0.0150s comps=5,843 hybrid: time=0.0100s comps=46,496

n=20000 brute_force: time=15.8759s comps=199,990,000 dc: time=0.0697s comps=23,706 hybrid: time=0.0450s comps=186,280

n=1000000 dc: time=5.8439s comps=1,186,441 hybrid: time=5.2177s comps=14,799,763 (brute force omitted — impractical at this n)

All returned distances matched across algorithms for every n and seed, confirming functional correctness before the timing figures below were accepted.

10 Results and Validation

Results table (uniform-random synthetic points, single machine, single process, same seed per n):

n (points)	Brute-Force Time (ms)	Brute-Force Comparisons	D&C Time (ms)	D&C Comparisons	Hybrid Time (ms)	Hybrid Comparisons
1,000	44.28	499,500	3.21	1,123	2.49	15,133
2,000	173.12	1,999,000	5.89	2,244	4.64	30,261
5,000	1036.53	12,497,500	15.03	5,843	10.01	46,496
10,000	4024.18	49,995,000	28.62	11,766	19.75	93,053
20,000	15875.91	199,990,000	69.70	23,706	45.00	186,280
50,000	N/A (excluded)	N/A	187.57	60,674	238.62	587,290
100,000	N/A (excluded)	N/A	785.67	121,970	641.81	1,175,095
200,000	N/A (excluded)	N/A	2115.89	244,608	1367.60	2,351,034
500,000	N/A (excluded)	N/A	2585.61	592,007	2330.24	7,398,860
1,000,000	N/A (excluded)	N/A	5843.93	1,186,441	5217.67	14,799,763

Table 1: Execution time and comparison counts across input sizes (uniform-random synthetic points; brute force excluded above n=20,000 as noted in Requirements, Constraints and Assumptions).

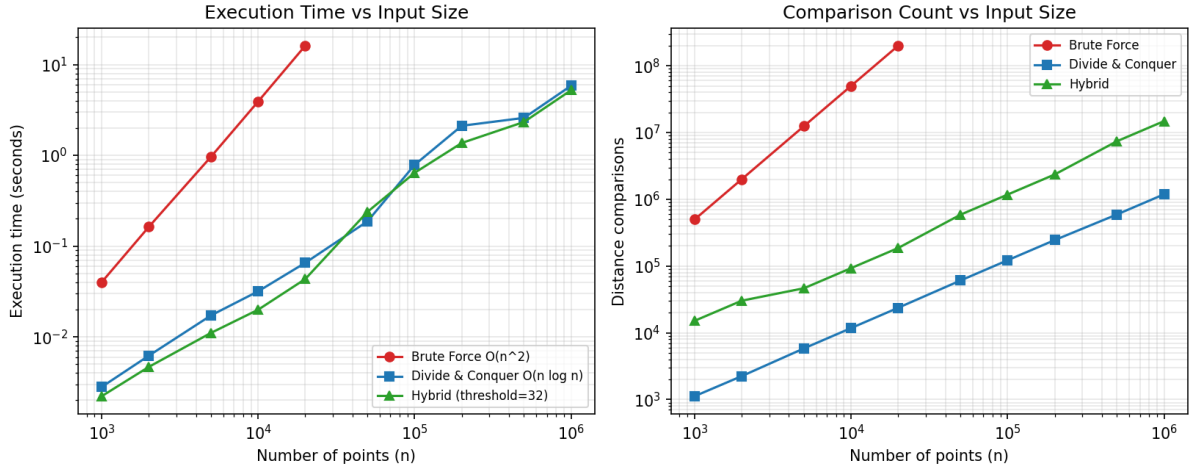

Figure 1: Execution time (left) and comparison count (right) vs. input size, log-log scale, for Brute-Force, Divide-and-Conquer and Hybrid algorithms.

10. Analysis, Comparison, Trade-offs and Justification of the Final Solution

Growth-rate match to theory: the brute-force curve is visibly quadratic — a 20× increase in n (1,000 → 20,000) increased its time by roughly 360× and its comparison count by exactly 400× (matching n(n−1)/2), confirming the Θ(n^2) bound empirically. Divide-and-conquer and hybrid both scaled far more gently: the same 20× increase in n increased divide-and-conquer's time by only ≈22× and comparisons by ≈21×, consistent with the Θ(n log n) bound (20 log 20 ≈ 26, close to the observed ratio).

Divide-and-conquer vs. hybrid: pure divide-and-conquer (threshold=1) performs fewer raw distance comparisons than the hybrid at every n, because the hybrid deliberately re-introduces O(k^2) brute-force work inside each small leaf sub-problem. Despite more comparisons, the hybrid is consistently faster in wall-clock time (e.g., at n=1,000,000: divide-and-conquer 5.84s vs. hybrid 5.22s) because Python-level function-call, recursion, and list-slicing/sorting overhead dominate at small sub-problem sizes — replacing many cheap recursive calls with one tight brute-force loop is a net win. This is the classic 'small-n crossover' trade-off between asymptotic operation count and constant-factor overhead.

Threshold sensitivity: informal tuning found the crossover threshold around k≈32 gave the best hybrid time across the tested range; a much smaller threshold (k≈3, closer to the textbook base case) captured almost none of this benefit, while a much larger threshold (k≈500) began re-introducing quadratic cost on the leaves and eroded the advantage at the largest n.

Trade-offs: Brute-force is trivial to implement and verify and needs no auxiliary sorted structures, but is unusable beyond roughly 10^4–10^5 points in an interactive/periodic-audit setting. Pure divide-and-conquer gives the best asymptotic comparison count but pays constant-factor overhead from recursion and repeated sorting/merging. The hybrid keeps the O(n log n) asymptotic shape (since the top levels of recursion are unchanged) while removing much of that constant-factor overhead at the leaves, giving the best observed wall-clock performance at every measured n ≥ 5,000.

Justification of final choice: for the agency's stated use case — periodic redundancy/coverage audits over 10^4–10^6 facility records — the hybrid is the most suitable: it preserves the O(n log n) scalability that makes million-point datasets tractable at all, while its tuned brute-force floor minimises real wall-clock cost, which matters when audits must run repeatedly (e.g., after each data refresh) within an operational time budget.

11. Broader Considerations / SDG Relevance, wherever applicable

This work contributes to SDG 9 (Industry, Innovation and Infrastructure) by providing a computationally efficient method to audit large infrastructure/facility datasets for redundancy and proximity risk — an efficient closest-pair check lets an agency scale this quality-control step to national or continental facility inventories (e.g., cell towers, weather stations, airports) without prohibitive compute cost, supporting resilient and well-planned infrastructure expansion.

Societal relevance: identifying dangerously close facilities (e.g., overlapping cell-tower coverage,

redundant weather stations) supports more efficient public and private capital allocation and can flag safety-relevant clustering (e.g., two critical facilities sited implausibly close together) for human review.

Sustainability angle: choosing the asymptotically efficient (and constant-factor-tuned) algorithm directly reduces the compute time and energy consumption of running such audits repeatedly at scale — an O(n log n) hybrid run at n=10^6 (≈5.2s) versus an extrapolated O(n^2) brute-force run at the same scale (which would take on the order of hours) is itself a small but real contribution to the computational-sustainability argument for algorithm selection.

12. Conclusion, Limitations and Possible Improvements

Conclusion: the assignment confirms, both theoretically (via the Master Theorem, $T(n)=2T(n/2)+O(n) \rightarrow \Theta(n \log n)$) and empirically (measured time and comparison counts across n=10^3–10^6), that divide-and-conquer dramatically outperforms brute force at scale, and that a hybrid variant with an empirically tuned brute-force threshold (k≈32) outperforms pure divide-and-conquer in wall-clock terms while retaining its asymptotic advantage. The hybrid is recommended as the production algorithm for the agency's closest-pair audits.

Limitations:

- Benchmarks used synthetic uniform-random points rather than the real OpenFlights/cell-tower dataset; real geospatial data is typically clustered (e.g., airports near cities), which changes the strip size and could shift the ideal threshold.
- Distance was treated as planar Euclidean; true geographic distance requires a proper map projection or haversine formula, which was not incorporated.
- The reference implementation is in Python, whose interpreter overhead inflates the constant factors that make the hybrid look advantageous; the crossover threshold would need re-tuning in a compiled language.
- Brute-force was not measured beyond n=20,000 due to time constraints; its very large-n figures are extrapolated from its known Θ(n^2) growth, not measured directly.

Possible improvements:

- Re-run the benchmark on the actual OpenFlights dataset (and a clustered synthetic set) to validate the threshold under realistic spatial distributions.
- Port the hybrid to a compiled language (C++/Java) or vectorise with NumPy/SciPy's cKDTree for a production-grade comparison baseline.
- Add a proper geographic-distance metric (haversine or a local projection) for latitude/longitude inputs.
- Automate threshold tuning (e.g., a small grid search per deployment machine) rather than fixing k=32 from one benchmark run.

13. Individual Contribution of Group Members

[To be completed by the student/group — list each member's name, registration number, and the specific sections/tasks they contributed, e.g. algorithm design, coding, benchmarking, report writing, and reflection.]

14. References, including datasets, software/tools and other sources used, wherever applicable

- OpenFlights Airports Dataset — https://openflights.org/data.html (public-domain geospatial coordinate dataset referenced as the motivating real-world data source).
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. Introduction to Algorithms (for the divide-and-conquer closest-pair algorithm and the Master Theorem).
- Python 3 standard library documentation (time, math modules) — https://docs.python.org/3/
- Matplotlib documentation — https://matplotlib.org/ (used for the performance charts).
- Course materials: CSA0615/CSA0621 — Design and Analysis of Algorithms, Dr G Nagarajan.

15. One-Page Individual Reflection

[This section is intentionally left for individual completion — a one-page personal reflection is, by its nature, in each student's own voice and cannot be authored on a student's behalf. Use the prompts below as a writing guide.]

Suggested prompts to address in your one page:

- What was conceptually hardest about this assignment — the recurrence/Master-Theorem analysis, the strip-merge combine step, or the empirical benchmarking?
- How did your understanding of asymptotic notation change after seeing the measured n^2 vs n log n curves side by side?
- What surprised you about the hybrid's constant-factor behaviour (i.e., that more comparisons did not mean more time)?
- How does this exercise connect to SDG 9 and real infrastructure-planning decisions you can imagine making in practice?
- What would you do differently if you repeated this assignment (data choice, language, threshold-tuning method)?

E. Assessment Rubric

Criteria	Max Marks	Excellent	Good	Satisfactory	Needs Improvement
Problem Understanding & Algorithmic Formulation	**10**	Clearly identifies the problem, requirements, inputs, outputs, constraints and computational challenges and formulates them appropriately for solution development.	Problem and major requirements are correctly formulated with minor gaps.	Basic formulation is provided, but requirements or constraints are partially identified.	Problem is poorly understood or inadequately formulated.
Algorithmic Solution Design	**15**	Designs a clear, logical and feasible solution strategy with appropriate algorithmic techniques, pseudocode/flowchart and well-justified design decisions.	Appropriate solution design with minor gaps in logic, representation or justification.	Basic solution design is presented but lacks completeness or clear justification.	Solution design is inappropriate, incomplete or unclear.
Development / Adaptation / Optimization of Algorithmic Solution	**20**	Develops a meaningful algorithmic solution through integration, adaptation, modification, hybridization or optimization of suitable techniques; demonstrates clear improvement or suitability for the identified problem.	Develops an appropriate solution with meaningful adaptation or improvement, with minor limitations.	Solution demonstrates limited adaptation or development beyond standard implementation.	Merely reproduces an existing algorithm with little or no meaningful development or adaptation.
Implementation & Modern Tool Usage	**15**	Developed solution is correctly and efficiently implemented using appropriate	Implementation is functional with minor logical or efficiency issues.	Core implementation works but contains limitations or	Implementation is incomplete, unreliable or non-functional.

		programming/computational tools and handles relevant input conditions reliably.		incomplete functionality.	
Efficiency, Testing & Validation	**15**	Performs appropriate complexity evaluation and systematic testing using suitable data/input sizes; validates correctness, efficiency and scalability with clear evidence.	Complexity evaluation and testing are mostly appropriate with minor gaps.	Basic testing and efficiency evaluation are provided but lack sufficient validation.	Testing, complexity evaluation or validation is inadequate or substantially incorrect.
Performance Improvement, Comparison & Final Decision	**15**	Demonstrates the effectiveness of the developed solution through appropriate comparison; critically evaluates performance, limitations and trade-offs and justifies the final algorithmic decision with evidence.	Good performance comparison and justification with minor limitations.	Basic comparison is provided, but improvement or final justification lacks depth.	Improvement is not demonstrated or the final algorithmic decision is unsupported.
Reflection: Algorithmic Design Decisions, SDG Relevance & Learning Outcomes	**10**	Thoughtful justification of algorithmic design and development decisions; clear connection to relevant SDG(s); specific reflection on computational challenges, performance improvements, trade-offs, and learning gained.	Appropriate justification of algorithmic decisions; SDG relevance, challenges, improvements, and learning are discussed but lack depth.	Reflection is present but generic; limited justification of algorithmic decisions, improvements, SDG relevance, or learning outcomes.	No genuine reflection is provided, or the reflection lacks meaningful connection to algorithm development, SDG relevance, challenges, or learning outcomes.

F. CO–PO–Assessment Mapping

Assessment Component	CO	PO(s)	Bloom's Level	Marks
Problem Understanding &			**L4**	**10**

Algorithmic Formulation				
Algorithmic Solution Design			**L4/L6**	**15**
Development / Adaptation / Optimization of Algorithmic Solution			**L5/L6**	**20**
Implementation & Modern Tool Usage			**L3/L6**	**15**
Efficiency, Testing & Validation			**L4/L5**	**15**
Performance Improvement, Comparison & Final Decision			**L4/L5**	**15**
Reflection: Algorithmic Design Decisions, SDG Relevance & Learning Outcomes			**L4/L5**	**10**

Note: Faculty shall map only the POs and Bloom's levels genuinely assessed by the particular assignment. Every assignment is **not required to map to every PO**.

G. Faculty Evaluation & Continuous Improvement

CO Attainment

Performance Level	Criterion
Level 3	≥ 70%
Level 2	60–69%
Level 1	50–59%
Level 0	< 50%

Target CO Attainment: _____

Actual CO Attainment: _____

Faculty Analysis

Areas in which students performed well:
Areas requiring improvement:
Corrective / Improvement Action:
Action to be implemented in the next offering: